PYTHON PROGRAMMING TASK DOCUMENTATION

Functions, Nested Functions, Closures, Decorators, Lambda, Filter and Map

A descriptive record of the eight Python programming tasks

Introduction

This document describes eight Python programming tasks focused on functions and commonly used functional programming techniques. The tasks cover salary calculation, nested functions, closures, decorators, lambda expressions, filtering, mapping, and combined processing of student marks. The documentation explains the purpose and working concept of each task without including source code, task questions, or program outputs.

Objectives

- Understand how functions receive values and return calculated results.
- Learn how nested functions organize related operations.
- Understand closures and how an inner function can retain values from an outer function.
- Learn how decorators add behaviour around an existing function.
- Use lambda expressions for short, anonymous operations.
- Apply filter() to select elements that satisfy a condition.
- Apply map() to transform every element in a collection.
- Combine these concepts to process and classify student marks.

Task 1: Function – Salary Calculation

Description

This task focuses on creating and using a function to calculate an employee's total salary. The function receives three salary components: basic salary, House Rent Allowance (HRA), and Dearness Allowance (DA). It combines these components and returns the total salary. The task demonstrates function parameters, arithmetic operations, return values, and function calls.

CODE:

```
def calculate_salary(salary,HRA,DA):
    return salary+HRA+DA
sal=int(input("Enter the salary:"))
HRD=int(input("Enter the HRD:"))
DA=int(input("Enter the DA:"))
print("TOTAL SALARY:",calculate_salary(sal,HRD,DA))
```

Task 2: Nested Function – Electricity Bill Calculator

Description

This task demonstrates the use of a function defined inside another function. The outer function accepts the number of electricity units consumed. The inner function performs the bill calculation using the consumed

units and the rate applied in the program. The outer function calls the inner function and returns the calculated bill amount.

CODE:

```
def calculate_bill(units):
    def calculate_units():
        bill=units*5
        return bill
    bill=calculate_units()
    return bill


unit=int(input("Enter the input:"))
print("Total bill =",calculate_bill(unit))
```

Task 3: Closure – Discount Calculator

Description

This task demonstrates a closure through a discount calculator. An outer function accepts a discount percentage and returns an inner function. The returned inner function accepts a product price and calculates the price after applying the discount. The inner function continues to use the discount value supplied to the outer function.

CODE:

```
def discount_generator(discount):
    def calculate(price):
        finalprice=price-(price*discount/100)
        return finalprice
    return calculate


amount=int(input("Enter the amount:"))
discount=int(input("Enter the discount:"))
cal=discount_generator(discount)
print("Thee final amount is ",cal(amount))
```

Task 4: Decorator – Login Verification

Description

This task introduces a decorator used for login verification. The decorator is designed to check whether a user is logged in before allowing a protected function, such as a dashboard function, to execute. When the user is authenticated, access to the dashboard is allowed. When the user is not authenticated, the application displays a message asking the user to log in first.

CODE:

```
def login_required(greet):
    print("Details procesing.....")
    flag=False
    def secverification():
        flag=True
        print("Second verification....")
        ref=greet(flag)


    return secverification
@login_required
def greet(flag):
    if flag:
        print("User login")
    else:
        print("User not login")
greet()
```

Task 5: Lambda – Product Price with GST

Description

This task uses a lambda expression to perform a short calculation on a product price. The calculation determines the GST amount at a rate of 18 percent and adds it to the original product price to obtain the price including GST.

CODE:

```
price=int(input("Enter the price:"))
fun=lambda price:price*18/100
```

```
print(fun(price))
```

Task 6: Filter – Number Filtering

Description

This task applies the filter() function with lambda expressions to select values from a given list of numbers. The list is examined using four conditions: selecting even numbers, selecting numbers greater than 50, selecting numbers divisible by 5, and selecting numbers between 20 and 80.

CODE:

```
arr=[12, 45, 67, 23, 89, 90, 34, 100, 15]

print(arr)

k=list(filter(lambda x:x%2==0,arr))

print("the even numbeers in the list are:",k)

k=list(filter(lambda x:x>50,arr))

print("Filter numbers greater than 50:",k)

k=list(filter(lambda x:x%5==0,arr))

print("Filter numbers divisible by 5:",k)

k=list(filter(lambda x:x>20 and x<80,arr))

print("Filter numbers between 20 and 80:",k)
```

Task 7: Map – Data Transformation

Description

This task uses the map() function with lambda expressions to transform every element in a collection. The operations include converting Celsius temperatures into Fahrenheit, converting numbers into their squares, converting characters in text to uppercase, and adding 100 to each number in a list.

CODE:

```
temp=[0, 25, 30, 37, 100]

print(temp)

k=list(map(lambda x:((x*9/5)+32),temp))

print("convert Celsius to Fahrenheit:",k)

k=list(map(lambda x:x*x,temp))
```

```
print("Convert all numbers into squares:",k)


string=input("Enter the string:")
k1=''.join(map(lambda x:x.upper(),string))
print("Convert all names into uppercase:",k1)
k=list(map(lambda x:x+100,temp))
print("Add 100 to each number in a list:",k)
```

Task 8: Combined Function Challenge – Student Marks Processing

Description

This task combines several Python concepts to process a collection of student marks. It includes calculating the average mark, arranging marks in descending order using a lambda-based sorting key, selecting marks above 60 with filter(), and converting marks into grades with map(). It also uses a closure to classify each mark as pass or fail based on a pass-mark threshold, and a decorator to display a processing-start message before the marks-processing function runs.

CODE:

```
def calavg(marks):
    sum=0
    for i in range(0,len(marks)):
        sum=sum+marks[i]
    return sum//len(marks)
marks = [35, 45, 67, 89, 90, 23, 56, 78, 99]
print("The average marks:",calavg(marks))
print("-----------------------------------------------------")
result = sorted(marks, key=lambda x: x, reverse=True)
print(result)
print("-----------------------------------------------------")
k=list(filter(lambda x:x>60,marks))
print("Use filter to find marks above 60:",k)
print("-----------------------------------------------------")
grade = list(map(lambda x:
    'A' if x >= 90 else
```

```
    'B' if x >= 80 else
    'C' if x >= 70 else
    'D' if x >= 60 else 'F', marks))

print("Use map to convert marks into grades:",grade)
print("-------------------------------------------------------")
def pass_fail_generator(pass_mark):
    def check(marks):
        if marks >= pass_mark:
            return "Pass"
        else:
            return "Fail"
    return check

check_result = pass_fail_generator(40)
result = list(map(check_result, marks))
print("Pass/Fail:", result)
print("-------------------------------------------------------")

def marks_decorator(fun):

    def inner():
        print("Marks processing started")
        fun()

    return inner
@marks_decorator
def process_marks():
    print("Processing student marks...")
process_marks()
```